



Advanced Python – Day 2

Constructors + Instance Variables



Goal: Understand how objects store data properly

Today = Real OOP clarity.

What is a Constructor?

- ⚡ A constructor is a special method in Python.
- </> It is named `__init__()`
- ▶ Runs automatically when an object is created.
- 🗄 Primarily used to initialize object data.

```
class Example: def __init__(self):  
    print("Constructor running!")
```



Basic Constructor Structure

1. Developer Code

We write a class and instantiate it normally:

```
class Person: def __init__(self, name, age): self.name = name self.age = age
# Creating an object p1 =
Person("Saurav", 21)
```

2. Internal Python execution

When you do `p1 = Person(...)`, Python internally executes:

```
Person.__init__(p1, "Saurav",
21)
```

✓ Constructor runs automatically!

Why Do We Use Constructors?

✗ Without Constructor

```
class Person: pass
p1 = Person()
p1.name = "Saurav"
p1.age = 21
```

This works, but...

- ✗ It's messy
- ✗ No control
- ✗ Risky in big systems

✓ With Constructor

Constructor makes object creation strictly structured.

- 🛡 Clean & Readable
- 🔒 Safe (Forces required data)
- 📦 Structured memory setup

Instance Variables

Instance variables are created using `self.variable_name`. They belong specifically to the object being created.

```
class Student: def __init__(self, name, marks): self.name = name self.marks = marks
```

 Each object stores its own data.

 Linked directly to `self`.



Object **Memory** Separation

Creating Multiple Objects

```
# From our Student class s1 =  
Student("Amit", 80) s2 =  
Student("Rahul", 90)
```

Two distinct objects exist in memory.

The Reality

≠ s1.marks ≠ s2.marks

🧠 Each object has separate memory space.

📄 Modifying one does not affect the other.

This is incredibly powerful.

Example 1: BankAccount Class

Class Definition

```
class BankAccount: def __init__(self,  
name, balance): self.name = name  
self.balance = balance
```

Execution

```
acc1 = BankAccount("Saurav",  
1000) acc2 = BankAccount("Rahul",  
2000) print(acc1.name, acc1.balance)  
print(acc2.name, acc2.balance) #  
Output: # Saurav 1000 # Rahul 2000
```


Example 2: Laptop Class

Class Definition

```
class Laptop: def __init__(self,  
brand, price): self.brand = brand  
self.price = price
```

Execution

```
l1 = Laptop("HP", 50000) l2 =  
Laptop("Dell", 60000) print(l1.brand)  
# HP print(l2.price) # 60000
```

Objects hold their own unique attributes.

What If We Forget **self**?

```
class Test: def __init__(name): # Missing  
self! name = name
```

✗ This causes an error.

Why?

Because Python automatically sends the object reference as the first argument when you create an object.

self is mandatory.



The Default Constructor

Even if you don't pass any external parameters, the constructor still runs automatically.

```
class Demo: def __init__(self): print("Object  
Created!") d1 = Demo()
```

- ⚙ Useful for setting default states.
- ▶ Execution is guaranteed.





Output Prediction

What is the output?

```
class Animal: def __init__(self,
name): self.name = name a1 =
Animal("Dog") a2 = Animal("Cat")
a1.name = "Lion" print(a1.name)
print(a2.name)
```

Correct Output

Lion Cat

Why?

Because each object has **separate memory**.
Modifying `a1` has zero impact on `a2`.

Practice Tasks

Hour 2: Practice & Deep Understanding

Task 1: Car



Create class **Car** with `brand`, `model`, `price`. Create 2 objects and print details.

Task 2: Employee



Create class **Employee** with `name`, `salary`. Create 3 employees and print them.

Task 3: Movie



Create class **Movie** with `title`, `rating`. Create objects and print ratings.

Task 4: Course



Create class **Course** with `course\_name`, `duration`. Create 2 objects.

||

Each object has its own memory space.
When you modify a1, it only changes a1, not
a2.

||

THAT'S THE POWER OF INSTANCE VARIABLES

Day 2 Summary



Deep understanding of Constructor

You now know why `__init__` exists and how it keeps code clean.



Instance Variables Mastery

Data belonging to individual objects.



self Clarity

Understanding the mandatory first parameter.



Object Memory Separation

Isolated instances that don't overwrite each other.



Congratulations!

You are now thinking like a real OOP developer.

Image Sources



https://media.istockphoto.com/id/1413990965/photo/glowing-python-programming-language-code-on-a-blue-digital-surface-with-a-sphere-grid-design.jpg?s=612x612&w=0&k=20&c=sC5Gcktg85_J6e50P5DGngrez1MEllqI5xrYKdI3sA4=

Source: www.istockphoto.com



https://miro.medium.com/v2/resize:fit:1400/1*WlO2Zqw5zVk-mvFLFfIREA.jpeg

Source: medium.com



https://static.vecteezy.com/system/resources/previews/015/286/179/non_2x/abstract-warning-and-warning-triangle-symbols-binary-red-background-hack-warning-concept-cyber-security-data-hacking-theft-prevention-on-red-and-black-background-vector.jpg

Source: www.vecteezy.com



https://eu-images.contentstack.com/v3/assets/blt07f68461ccd75245/bltce0b55d3d80839f9/6627725aa292b70ef94c557f/blue_gears_6.jpg?width=1280&auto=webp&quality=80&format=jpg&disable=upscale

Source: www.itprotoday.com
